

Production-Grade Kubernetes SRE Platform

Table of Contents

1 Production-Like Kubernetes SRE Lab Project.....	4
2. Production Architecture.....	4
3. Recommended GitHub Repository Structure.....	5
3.1 Arborescence.....	5
3.3 Créer le dossier projet.....	6
3.4 créer la structure initiale :.....	6
3.5 initialiser git.....	6
3.6 créer readme minimal.....	6
3.7 premier commit.....	7
3.8 Créer le repo github.....	7
3.9 Connecter le repo.....	8
3.10 premier push.....	8
4. Incremental Implementation Phases.....	8
4.1 Phase 1 — Cluster + Ingress.....	8
4.1.1 Install k3d.....	8
4.1.2 Create Cluster.....	8
4.1.3 Traefik or gninx-ingress.....	9
4.1.4 Créer le namespace :.....	10
4.2 Phase 2 — Monitoring Stack.....	10
Add Helm Repositories.....	10
Install kube-prometheus-stack.....	10
Verify:.....	10
4.3 Phase 3 — Logging Stack.....	11
Install Loki Stack.....	11
Verify:.....	11
4.4 Phase 4 — Tester Grafana.....	11
4.5 Phase 5— Tester Prometheus.....	12
4.6 Phase 6 — GitOps with ArgoCD.....	12
Install ArgoCD :.....	12
Expose UI:.....	12
Get password:.....	12
Vérifier :.....	13
4. Phase 7 — Autoscaling + Load Testing.....	13

Install Metrics Server.....	13
Vérifier le serveur de métriques :.....	13
vérifier les métriques nodes :.....	13
vérifier le métriques pods :.....	14
5 Deployment.....	14
5.1 App/deployment.yaml.....	14
5.1.1 Deployer application :.....	15
5.1.2 verifier pods :.....	15
5.2 App/service.yaml.....	15
5.2.1 Déployer service.....	16
5.2.2 verifier service :.....	16
5.3 App/ingress.yaml.....	16
5.3.1 Déployer ingress.....	17
5.3.2 verifier ingress.....	17
5.4 App/hpa.yaml.....	17
5.4.1 Créer le fichier app/hpa.yaml.....	17
averageUtilization: 5.....	18
5.4.2 Configuration du scale down.....	18
5.4.3 Déploiement du .yaml :.....	18
5.4.4 Vérifier HPA :.....	19
5.4.5 tester application :.....	19
5.4.6 Vérifier probes :.....	19
5.5 commit git.....	20
6. ArgoCD Application YAML.....	20
Argocd/application.yaml	20
6.1 Vérification de la syntaxe.....	21
6.2 Deployer :.....	21
6.3 Vérification 1 :.....	21
6.4 vérification 2 :.....	22
6.5 Vérification en cours de lab de l'état de Argo.....	22
7 Centralisation des logs (Loki/Grafana).....	23
7.01 Objectifs :.....	23
7.02 Résultats :.....	23
7.1 Architecture Loki.....	23
7.2 Installation.....	23
7.5 Vérifications.....	24
7.6 logging/promtail-values.yaml.....	25

Vérifier promtail.....	25
7.8 Générer des logs.....	26
7.9 Déployer.....	27
7.10 Vérifier les log K8s.....	27
7.11 Vérifier que Loki reçoit les logs.....	27
7.12 Tester les logs dans Grafana.....	28
7.13 Erreur frequente avec Grafana/loki.....	28
8 État du cluster au repos (avant charge).....	29
8.1 État du cluster avant les tests de charge.....	29
8.2 k9s – vue pods.....	31
8.3 Grafana – Vue cluster.....	32
9 Load Testing.....	32
9.1 Test 1 (K6).....	32
9.1.1 Installer K6.....	32
9.1.2 fichier de test.....	32
9.1.3 Run:.....	33
9.1.4 Observer scaling :.....	33
9.2 Test 2 (hey).....	35
9.3 Test K6 plus agressif.....	38
9.4 Tests Logs Lki.....	39
10 Incident Simulation.....	39
10.1 Ingress Failure.....	39

1 Production-Like Kubernetes SRE Lab Project

This lab is designed as a **portfolio-quality SRE / DevOps / Platform Engineering project** for:

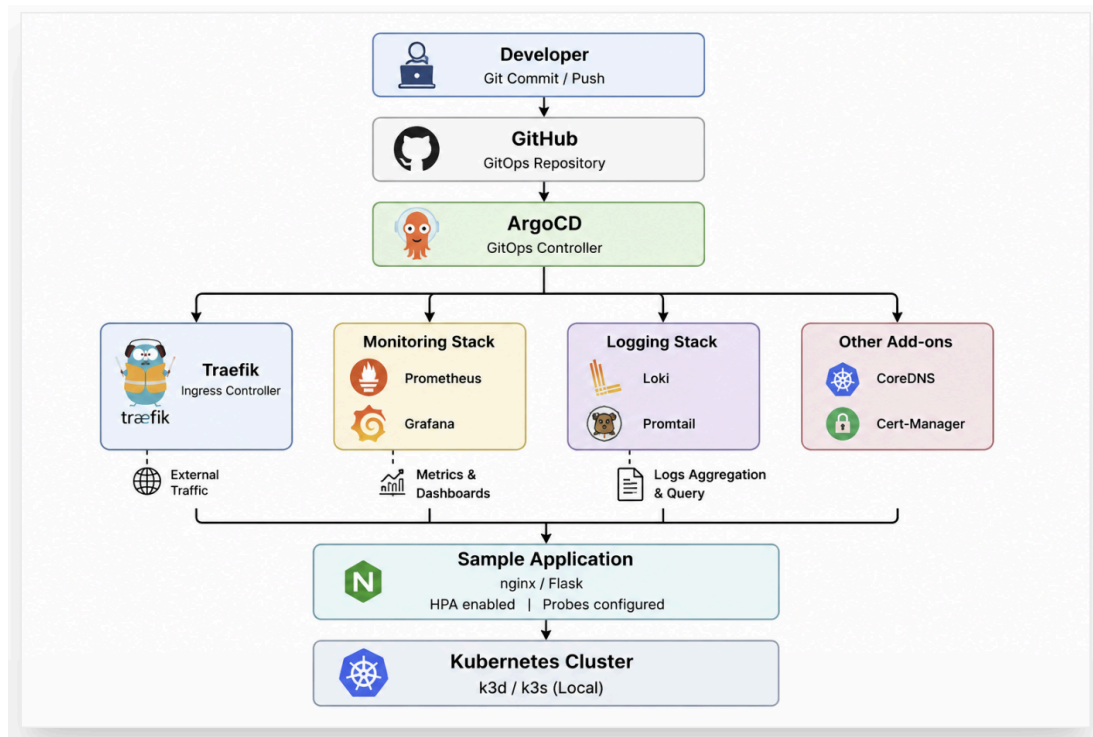
- Senior SRE recruiters
- DevOps Engineer interviews
- Cloud Platform Engineer portfolios
- GitHub showcase projects

It simulates a realistic production Kubernetes environment locally using:

- k3d (recommended) or kind
- GitOps with Argo CD
- Monitoring with Prometheus + Grafana
- Logging with Loki + Promtail

- ingress-nginx
- HPA autoscaling
- Load testing
- Incident simulation
- Production-grade Kubernetes manifests

2. Production Architecture



3. Recommended GitHub Repository Structure

3.1 Arborescence


```

henri@Henri:~/labs/sre-kubernetes-lab$ tree
.
├── README.md
├── app
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   └── service.yaml
├── argo.test
├── argocd
│   └── application.yaml
├── cluster
│   └── k3d-config.yaml
├── docs
│   └── readme.md
├── incidents
├── ingress
├── load-testing
│   └── k6-loadtest.js
├── logging
│   ├── log-generator.yaml
│   ├── loki-stack.yaml
│   └── promtail-values.yaml
├── logs
│   ├── kubernetes-logs-loki-0-logging-tail-100.log
│   └── log-loki-promtail-84cdl
├── monitoring
├── screenshots
│   └── readme.md
└── scripts

13 directories, 16 files
henri@Henri:~/labs/sre-kubernetes-lab$

```

3.3 Créer le dossier projet

```

mkdir sre-kubernetes-lab
cd sre-kubernetes-lab

```

3.4 créer la structure initiale :

```

mkdir -p \cluster \monitoring \logging \argocd \app \load-testing \incidents \scripts

```

3.5 initialiser git

```

git init

```

3.6 créer readme minimal

```

touch README.md

```

3.7 premier commit

```

git add .
git commit -m "Initial project structure"

```

```

henri@Henri:~/labs/sre-kubernetes-lab$ pwd
/home/henri/labs/sre-kubernetes-lab
henri@Henri:~/labs/sre-kubernetes-lab$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/henri/labs/sre-kubernetes-lab/.git/
henri@Henri:~/labs/sre-kubernetes-lab$ touch README.md
henri@Henri:~/labs/sre-kubernetes-lab$ dir
README.md app argocd cluster incidents ingress load-testing logging monitoring scripts
henri@Henri:~/labs/sre-kubernetes-lab$ git add .
henri@Henri:~/labs/sre-kubernetes-lab$ git commit -m "Initial project structure"
[master (root-commit) 4b78dd5] Initial project structure
5 files changed, 92 insertions(+)
create mode 100644 README.md
create mode 100644 app/deployment.yaml
create mode 100644 app/hpa.yaml
create mode 100644 app/ingress.yaml
create mode 100644 app/service.yaml
henri@Henri:~/labs/sre-kubernetes-lab$

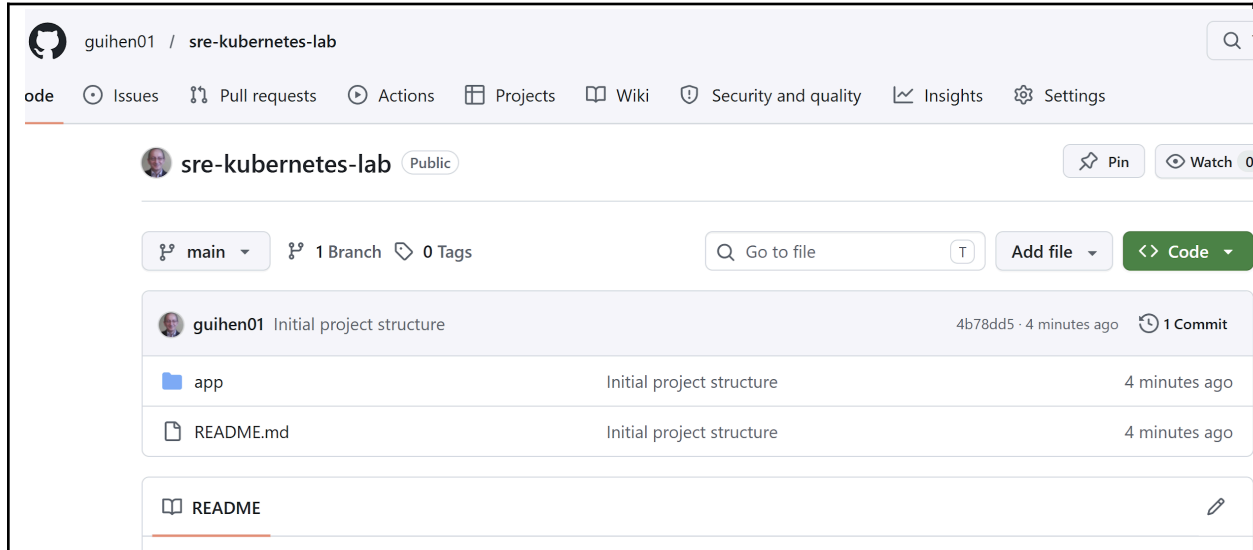
```

3.8 Créer le repo github

Nom du repository : sre-kubernetes-lab

Se connecter sur votre compte github

Créer le repository :



3.9 Connecter le repo

git remote add origin https://github.com/<username>/sre-kubernetes-lab.git

3.10 premier push

git branch -M main

git push -u origin main

4. Incremental Implementation Phases

4.1 Phase 1 — Cluster + Ingress

4.1.1 Install k3d

```
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
```

4.1.2 Create Cluster

```
k3d cluster create sre-lab \  
  --agents 2 \  
  -p "8080:80@loadbalancer" \  
  -p "8443:443@loadbalancer"
```

Autre option pour créer le cluster : `k3d cluster create --config cluster/k3d-config.yaml`

Cela créera 1 pod de type server et 2 pods de type workers

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get nodes  
NAME                                STATUS    ROLES    AGE   VERSION  
k3d-sre-lab-agent-0                 Ready     <none>    80s   v1.31.5+k3s1  
k3d-sre-lab-agent-1                 Ready     <none>    80s   v1.31.5+k3s1  
k3d-sre-lab-server-0                Ready     control-plane,master 84s   v1.31.5+k3s1
```

En vérifiant les clusters, on voit que le nouveau cluster a 1 pod server (1/1) qui est présentement actif et 2 pods de type workers (2/2) qui sont actifs.

```
henri@Henri:~/labs/sre-kubernetes-lab$ k3d cluster list  
NAME          SERVERS  AGENTS  LOADBALANCER  
gitops-lab    0/1      0/0     true  
henri         0/1      0/0     true  
ingress-lab   0/1      0/0     true  
platform-lab 0/1      0/0     true  
sre-lab       1/1      2/2     true  
henri@Henri:~/labs/sre-kubernetes-lab$
```

Avec k9s :

```
Context: k3d-sre-lab [RW]
Cluster: k3d-sre-lab
User:    admin@k3d-sre-lab
K9s Rev: v0.50.18
K8s Rev: v1.31.5+k3s1
CPU:     0%
MEM:     8%

<e>  Cordon          <u>  Uncordon
<ctrl-d> Delete          <y>  YAML
<d>   Describe
<f>   Drain
<e>   Edit
<?>  Help
```



NAME↑	STATUS	ROLE	TAINTS	VERSION	PODS	CPU	CPU/A	%CPU	MEM	MEM/A	%MEM	GPU/A	/C
k3d-sre-lab-agent-0	Ready	<none>	0	v1.31.5+k3s1	10	139	16000	0	1079	13832	7	n/a	/a
k3d-sre-lab-agent-1	Ready	<none>	0	v1.31.5+k3s1	11	120	16000	0	717	13832	5	n/a	/a
k3d-sre-lab-server-0	Ready	control-plane,master	0	v1.31.5+k3s1	9	187	16000	1	1742	13832	12	n/a	/a

4.1.3 Traefik or gninx-ingress

If traefik no installed then Install ingress-nginx (seulement si trafik n'est pas installé)

kubectl apply -f

<https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml>

Verify: kubectl get pods -n ingress-nginx

Else

kubectl get pods -n kube-system

on vérifie que traefik est installé. Ce qui est le cas dans notre lab.

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-ccb96694c-94jd6             1/1     Running   1 (15m ago) 14h
local-path-provisioner-5cf85fd84d-2dxrt 1/1     Running   1 (15m ago) 14h
metrics-server-5985cbc9d7-fmh7c      1/1     Running   1 (15m ago) 14h
svclb-traefik-e0711a08-6sbg2         2/2     Running   8 (15m ago) 3d
traefik-5d45fc8cc9-bl7jg            1/1     Running   1 (15m ago) 14h
henri@Henri:~/labs/sre-kubernetes-lab$
```

Vérifier les IngressClass :

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get ingressclass
NAME          CONTROLLER          PARAMETERS  AGE
traefik       traefik.io/ingress-controller  <none>      3d1h
henri@Henri:~/labs/sre-kubernetes-lab$
```

endif

4.1.4 Créer le namespace :

kubectl create namespace demo-app

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get ns
NAME          STATUS  AGE
default       Active  33m
demo-app      Active  8s
kube-node-lease Active  33m
kube-public   Active  33m
kube-system   Active  33m
henri@Henri:~/labs/sre-kubernetes-lab$
```

4.2 Phase 2 — Monitoring Stack

Add Helm Repositories

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Install kube-prometheus-stack

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
```

Verify:

```
kubectl get pods -n monitoring
```

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0  2/2     Running   0          24s
monitoring-grafana-74598f477b-c8sfj  2/3     Running   0          28s
monitoring-kube-prometheus-operator-6c8c5bb9b-vmz7d    1/1     Running   0          28s
monitoring-kube-state-metrics-64485fd97-8bmjv          1/1     Running   0          28s
monitoring-prometheus-node-exporter-hvdc8              1/1     Running   0          28s
monitoring-prometheus-node-exporter-mxplk              1/1     Running   0          28s
monitoring-prometheus-node-exporter-qkvkk              1/1     Running   0          28s
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   0          24s
henri@Henri:~/labs/sre-kubernetes-lab$
```

4.3 Phase 3 — Logging Stack

Install Loki Stack

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

```
helm install loki grafana/loki-stack \
  --namespace logging \
  --create-namespace
```

Verify:

```
kubectl get pods -n logging
```

```

henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get ns
NAME                STATUS    AGE
default              Active   44m
demo-app             Active   11m
kube-node-lease      Active   44m
kube-public          Active   44m
kube-system          Active   44m
logging              Active   16s
monitoring           Active   3m15s
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n logging
NAME                READY    STATUS    RESTARTS   AGE
loki-0              0/1      Running   0           30s
loki-promtail-84cdl 1/1      Running   0           30s
loki-promtail-sq7r2 1/1      Running   0           30s
loki-promtail-tvt9v 1/1      Running   0           30s
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$

```

kubectl get svc -n monitoring

```

HXSAKWE0TDjsT0yhenri@Henri:~/labs/sre-kubernetes-lab$ kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
alertmanager-operated              ClusterIP    None           <none>          9093/TCP,9094/TCP,9094/UDP            21m
monitoring-grafana                 ClusterIP    10.43.232.73   <none>          80/TCP                                21m
monitoring-kube-prometheus-alertmanager ClusterIP    10.43.4.217    <none>          9093/TCP,8080/TCP                     21m
monitoring-kube-prometheus-operator ClusterIP    10.43.109.93   <none>          443/TCP                                21m
monitoring-kube-prometheus-prometheus ClusterIP    10.43.5.194    <none>          9090/TCP,8080/TCP                     21m
monitoring-kube-state-metrics       ClusterIP    10.43.122.255  <none>          8080/TCP                                21m
monitoring-prometheus-node-exporter ClusterIP    10.43.73.242   <none>          9100/TCP                                21m
prometheus-operated                 ClusterIP    None           <none>          9090/TCP                                21m
henri@Henri:~/labs/sre-kubernetes-lab$

```

4.4 Phase 4 — Tester Grafana

```

kubectl port-forward svc/monitoring-grafana \
-n monitoring 3000:80

```

Pour récupérer le mot de passe admin de Grafana installé avec le chart kube-prometheus-stack :

```

kubectl get secret -n monitoring monitoring-grafana \
-o jsonpath="{.data.admin-password}" | base64 --decode ; echo

```

<http://localhost:3000>

4.5 Phase 5— Tester Prometheus

```

kubectl port-forward svc/monitoring-kube-prometheus-prometheus \
-n monitoring 9090

```

4.6 Phase 6 — GitOps with ArgoCD

Install ArgoCD :

```

kubectl create namespace argocd
kubectl apply -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

```

Expose UI:

```
kubect port-forward svc/argocd-server -n argocd 8081:443
```

Get password:

```
kubect -n argocd \  
get secret argocd-initial-admin-secret \  
-o jsonpath="{.data.password}" | base64 -d
```

Vérifier :

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n argocd
NAME                                READY    STATUS    RESTARTS    AGE
argocd-application-controller-0     1/1     Running   0           4h12m
argocd-applicationset-controller-866cbd67bd-72n4g  1/1     Running   19 (2m2s ago)  4h12m
argocd-dex-server-cdd7bd88f-p8wdr    1/1     Running   0           4h12m
argocd-notifications-controller-5b59cc856f-xnhqm  1/1     Running   0           4h12m
argocd-redis-548bf6d4d7-snjkh        1/1     Running   0           4h12m
argocd-repo-server-869988b885-f96np   1/1     Running   0           4h12m
argocd-server-56b6d45cd6-fg2n9       1/1     Running   0           4h12m
henri@Henri:~/labs/sre-kubernetes-lab$
```

4. Phase 7 — Autoscaling + Load Testing

Install Metrics Server

```
kubectl apply -f \  
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml  
!
```

Vérifier le serveur de métriques :

```
kubectl get deployment metrics-server -n kube-system
```

```
henri@Henri:~/labs/sre-kubernetes-lab$  
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get deployment metrics-server -n kube-system
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
metrics-server 1/1      1             1            7h41m
henri@Henri:~/labs/sre-kubernetes-lab$
```

On doit voir quelque chose comme : metrics-server-xxxxx Running

```
kubectl get apiservices | grep metrics
```

```
henri@Henri:~/labs/sre-kubernetes-lab$  
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get apiservices | grep metrics
v1beta1.metrics.k8s.io    kube-system/metrics-server    True    7h42m
henri@Henri:~/labs/sre-kubernetes-lab$  
henri@Henri:~/labs/sre-kubernetes-lab$
```

on doit voir : metrics.k8s.io et True

exemple : v1beta1.metrics.k8s.io kube-system/metrics-server True

vérifier les métriques nodes :

kubectl top nodes

On doit obtenir :

NAME	CPU(cores)	CPU%	MEMORY(bytes)
k3d-sre-lab-agent	120m	6%	800Mi

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl top nodes
NAME                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
k3d-sre-lab-agent-0  134m        0%       1094Mi          7%
k3d-sre-lab-agent-1  112m        0%       678Mi           4%
k3d-sre-lab-server-0 175m        1%       1746Mi          12%
henri@Henri:~/labs/sre-kubernetes-lab$
```

vérifier le métriques pods :

kubectl top pods -A

on doit voir CPU/MEM des pods.

5 Deployment

5.1 App/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
  namespace: demo-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
```



```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"

livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 10
  periodSeconds: 5

readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 3
```

5.1.1 Deployer application :

```
kubectl apply -f app/deployment.yaml
```

5.1.2 verifier pods :

```
kubectl get pods -n demo-app
```

```
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl apply -f app/deployment.yaml
deployment.apps/demo-app created
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n demo-app
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-58787c9f9d-5xjz6          1/1     Running   0           20s
demo-app-58787c9f9d-vtqgd          1/1     Running   0           20s
henri@Henri:~/labs/sre-kubernetes-lab$
```

5.2 App/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: demo-app
  namespace: demo-app
```

```
spec:
  selector:
    app: demo-app
  ports:
  - port: 80
    targetPort: 80
```

5.2.1 Déployer service

```
kubectl apply -f app/service.yaml
```

5.2.2 verifier service :

```
kubectl get svc -n demo-app
```

```
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl apply -f app/service.yaml
service/demo-app created
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get svc -n demo-app
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
demo-app     ClusterIP   10.43.79.172  <none>       80/TCP     9s
henri@Henri:~/labs/sre-kubernetes-lab$
```

5.3 App/ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-app
  namespace: demo-app
spec:
  ingressClassName: traefik
  rules:
  - host: demo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: demo-app
            port:
              number: 80
```

5.3.1 Déployer ingress

```
kubectl apply -f app/ingress.yaml
```

5.3.2 verifier ingress

```
kubectl get ingress -n demo-app
```

```
henri@Henri:~/labs/sre-kubernetes-lab$  
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl apply -f app/ingress.yaml  
ingress.networking.k8s.io/demo-app created  
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get ingress -n demo-app  
NAME          CLASS    HOSTS          ADDRESS          PORTS    AGE  
demo-app      traefik  demo.local     172.21.0.3,172.21.0.4,172.21.0.5  80      22s  
henri@Henri:~/labs/sre-kubernetes-lab$
```

5.4 App/hpa.yaml

5.4.1 Créer le fichier app/hpa.yaml

```
apiVersion: autoscaling/v2  
kind: HorizontalPodAutoscaler  
metadata:  
  name: demo-app-hpa  
  namespace: demo-app  
spec:  
  scaleTargetRef:  
    apiVersion: apps/v1  
    kind: Deployment  
    name: demo-app  
  
  minReplicas: 2  
  maxReplicas: 10  
  
  behavior:  
    scaleDown:  
      stabilizationWindowSeconds: 30  
  
    policies:  
      - type: Percent  
        value: 50  
        periodSeconds: 15  
  
  metrics:  
    - type: Resource  
      resource:  
        name: cpu  
        target:
```

```
type: Utilization
```

```
averageUtilization: 5
```

5.4.2 Configuration du scale down

Analyse de la partie de code, liée au scale down , dans le fichier précédent : hpa.yaml :

```
behavior:
scaleDown:
stabilizationWindowSeconds: 30
policies:
- type: Percent
value: 50
periodSeconds: 15
```

Ici :

Kubernetes attend 30 s

puis peut supprimer maximum 50% des pods toutes les 15 s

Exemple :

10 pods

après 30 s → passe à 5

15 s plus tard → 3

puis 2

Cette partie de code est importante, car au début elle, n'était pas présente et le cluster ne faisait pas vraiment de scale down visible, le nombre de pods restait au maximum de 10

5.4.3 Déploiement du .yaml :

```
kubectl apply -f app/hpa.yaml
```

5.4.4 Vérifier HPA :

```
kubectl get hpa -n demo-app
```

```
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl apply -f app/hpa.yaml
horizontalpodautoscaler.autoscaling/demo-app-hpa created
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get hpa -n demo-app
NAME          REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
demo-app-hpa  Deployment/demo-app  cpu: 1%/50%    2         10        2         14s
henri@Henri:~/labs/sre-kubernetes-lab$
```

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get hpa -n demo-app
NAME          REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
demo-app-hpa  Deployment/demo-app  cpu: 1%/50%    2         10        2         21h
henri@Henri:~/labs/sre-kubernetes-lab$
```

5.4.5 tester application :

curl <http://localhost:8080>

OU BIEN

curl -H "Host: demo.local" <http://localhost:8080>

```
henri@Henri:/mnt/c/Users/henri$ curl -H "Host: demo.local" http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
```

5.4.6 Vérifier probes :

kubectl describe pod -n demo-app <pod-name>

chercher :

Liveness probe succeeded

Readiness probe succeeded

```
henri@Henri:/mnt/c/Users/henri$ kubectl get pods -n demo-app
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-58787c9f9d-5xjz6          1/1     Running   0           10m
demo-app-58787c9f9d-vtqgd          1/1     Running   0           10m
henri@Henri:/mnt/c/Users/henri$
henri@Henri:/mnt/c/Users/henri$ kubectl describe pod -n demo-app demo-app-58787c9f9d-5xjz6
Name:                                demo-app-58787c9f9d-5xjz6
Namespace:                           demo-app
Priority:                              0
Service Account:                      default
Node:                                 k3d-sre-lab-agent-0/172.21.0.4
Start Time:                           Fri, 08 May 2026 19:35:28 -0400
Labels:                               app=demo-app
                                      pod-template-hash=58787c9f9d
Annotations:                           <none>
Status:                               Running
```

5.5 commit git

faire un commit git afin que les fichiers locaux .yaml soient poussés vers le repository github

git add .

git commit -m "Deploy stack"

git push

6. ArgoCD Application YAML

Argocd/application.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
```

```
name: demo-app
namespace: argocd

spec:
  project: default

source:
  repoURL: https://github.com/yourname/sre-kubernetes-lab.git
  targetRevision: HEAD
  path: app

destination:
  server: https://kubernetes.default.svc
  namespace: demo-app

syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

6.1 Vérification de la syntaxe

On vérifie la syntaxe du .yaml avant de le déployer. On utilise la commande pour faire un dry-run, une simulation de déploiement

kubectl apply --dry-run=client -f app/hpa.yaml

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl apply --dry-run=client -f app/hpa.yaml
horizontalpodautoscaler.autoscaling/demo-app-hpa configured (dry run)
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$
```

On doit voir :

horizontalpodautoscaler.autoscaling/demo-app-hpa created (dry run)

6.2 Deployer :

kubectl apply -f argocd/application.yaml

6.3 Vérification 1 :

kubectl get applications -n argocd

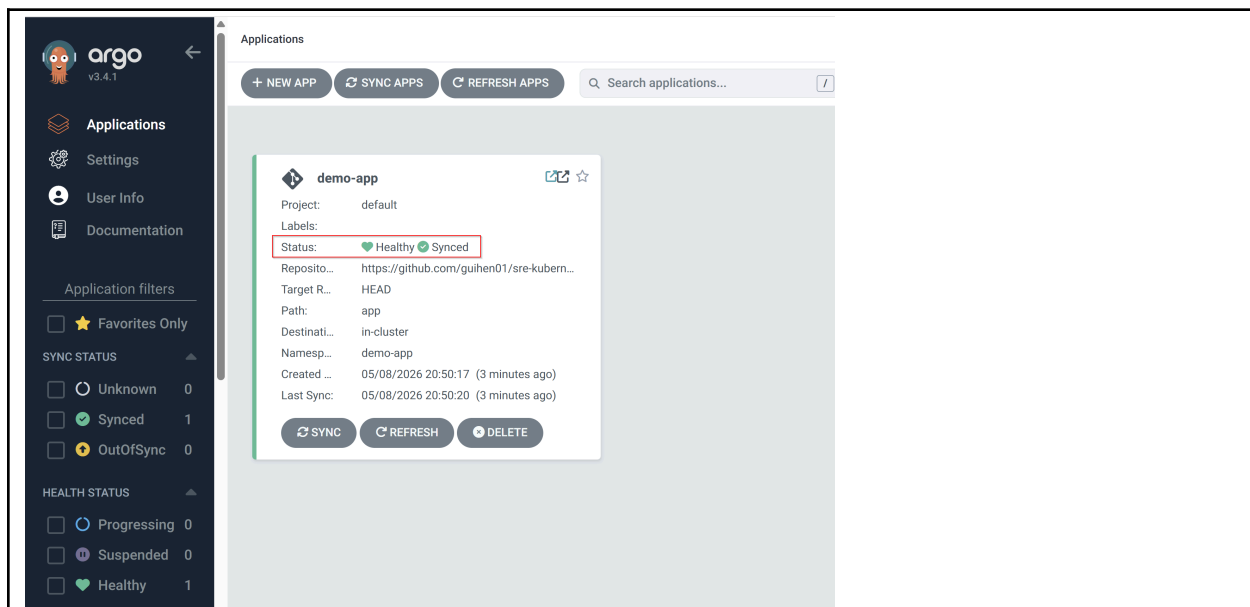
```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl apply -f argocd/application.yaml
application.argoproj.io/demo-app created
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get applications -n argocd
NAME          SYNC STATUS   HEALTH STATUS
demo-app      Synced        Healthy
henri@Henri:~/labs/sre-kubernetes-lab$
```

On doit obligatoirement voir s'afficher : Synced et Healthy

Si ce n'Est pas le cas, souvent la cause ets le .yaml mal configuré .
 Réviser le .yaml (voir la commande au paragraphe eprécédent : 6.1)
 Corriger la syntaxe .
 Puis pousser vers le repo git, à nouveau.
 git add .
 git commit -m "Fix HPA YAML indentation"
 git push
 revérifier ensuite qu'on est à nouveau dans l'état synced

6.4 vérification 2 :

Puis : aller dans Argo UO : <https://localhost:8081>
 Vérifier que : Healthy et Synced



6.5 Vérification en cours de lab de l'état de Argo

Toujours vérifier si argo est dans l'état synced
KubectI get applications -n argocd

On doit voir s'Afficher : synced healthy

Si ce n'est pas le cas , investiguer avec la commande :
kubectI describe application demo-app -n argocd

Vérifier dans la liste qui s'affiche les erreurs éventuelles

Corriger

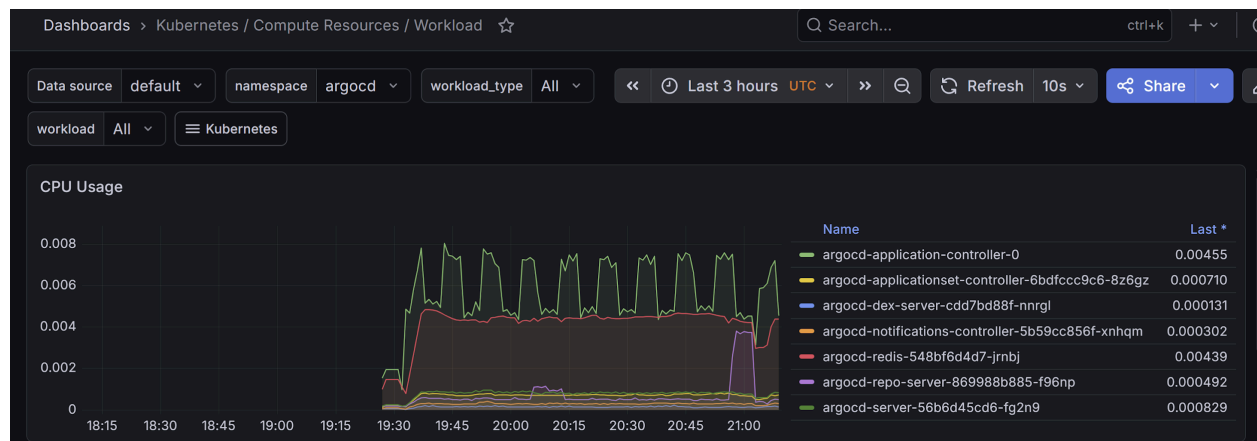
Redeployer avec git

Si nécessaire , Forcer le refresh de argocd : 9efface la mémoire cache , force la réécriture

```
kubectl annotate application demo-app \  
-n argocd \  
argocd.argoproj.io/refresh=hard --overwrite
```

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl annotate application demo-app \  
-n argocd \  
argocd.argoproj.io/refresh=hard --overwrite  
application.argoproj.io/demo-app annotated  
henri@Henri:~/labs/sre-kubernetes-lab$  
henri@Henri:~/labs/sre-kubernetes-lab$  
henri@Henri:~/labs/sre-kubernetes-lab$
```

6.6. Grafana – Vue argocd



7 Centralisation des logs (Loki/Grafana)

7.01 Objectifs :

Mettre en place une centralisation des logs Kubernetes avec :

Loki

Grafana

Promtail

7.02 Résultats :

L'intégration complète n'a pas été finalisée dans l'environnement local k3d/k3s en raison de problèmes de compatibilité et de configuration entre Loki et Grafana.

Ce qui a été testé :

déploiement Helm de Loki

datasource Grafana

collecte des logs pods

tests d'ingestion
Difficultés rencontrées
incompatibilités de versions
datasource non fonctionnelle
problèmes d'ingestion des logs
limitations environnement local k3d

7.1 Architecture Loki

Pods
↓
stdout/stderr
↓
Promtail
↓
Loki
↓
Grafana

7.2 Installation

```
helm install loki grafana/loki \
--namespace logging \
--create-namespace \
--set deploymentMode=SingleBinary \
--set singleBinary.replicas=1 \
--set backend.replicas=0 \
--set read.replicas=0 \
--set write.replicas=0 \
--set gateway.enabled=false \
--set lokiCanary.enabled=false \
--set test.enabled=false \
--set loki.auth_enabled=false \
--set loki.useTestSchema=true \
--set minio.enabled=true \
--set loki.storage.bucketNames.chunks=chunks \
--set loki.storage.bucketNames.ruler=ruler \
--set loki.storage.bucketNames.admin=admin
```

```
henri@henri:~/labs/sre-kubernetes-lab$ helm install loki grafana/loki \
--namespace logging \
--create-namespace \
--set deploymentMode=SingleBinary \
--set singleBinary.replicas=1 \
--set backend.replicas=0 \
--set read.replicas=0 \
--set write.replicas=0 \
--set gateway.enabled=false \
--set lokiCanary.enabled=false \
--set test.enabled=false \
--set loki.auth_enabled=false \
--set loki.useTestSchema=true \
--set minio.enabled=true \
--set loki.storage.bucketNames.chunks=chunks \
--set loki.storage.bucketNames.ruler=ruler \
--set loki.storage.bucketNames.admin=admin
```

A l'issue de l'installation, on voit le message : qui donne : <http://loki.logging.svc.cluster.local:3100/>

```

curl -H "Content-Type: application/json" -XPOST -s "http://127.0.0.1:3100/loki/api/v1/push" \
--data-raw '{"streams": [{"stream": {"job": "test"}, "values": [{"date +%s)000000000", "fizzbuzz"}]}]'
\

Then verify that Loki did receive the data using the following command:
\
curl "http://127.0.0.1:3100/loki/api/v1/query_range" --data-urlencode 'query={job="test"}' | jq .data.result
\

*****
Connecting Grafana to Loki
*****

If Grafana operates within the cluster, you'll set up a new Loki datasource by utilizing the following URL:
http://loki.logging.svc.cluster.local:3100/
henri@Henri:~/labs/sre-kubernetes-lab$

```

7.5 Vérifications

kubectl get pods -n logging

```

henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n logging
NAME                READY   STATUS    RESTARTS   AGE
loki-0               2/2     Running   0           8m9s
loki-chunks-cache-0  2/2     Running   0           8m9s
loki-minio-0         1/1     Running   0           8m9s
loki-results-cache-0 2/2     Running   0           8m9s

```

7.6 logging/promtail-values.yaml

```

helm install promtail grafana/promtail \
--namespace logging \
--set "config.clients[0].url=http://loki:3100/loki/api/v1/push"

```

```

henri@Henri:~/labs/sre-kubernetes-lab$ helm install promtail grafana/promtail \
--namespace logging \
--set "config.clients[0].url=http://loki:3100/loki/api/v1/push"
WARNING: This chart is deprecated
NAME: promtail
LAST DEPLOYED: Sat May  9 16:48:42 2026
NAMESPACE: logging
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
*****
Welcome to Grafana Promtail
Chart version: 6.17.1
Promtail version: 3.5.1
*****

Verify the application is working by running these commands:
* kubectl --namespace logging port-forward daemonset/promtail 3101
* curl http://127.0.0.1:3101/metrics
henri@Henri:~/labs/sre-kubernetes-lab$

```

Vérifier promtail

kubectl --namespace logging port-forward daemonset/promtail 3101

puis :

curl <http://127.0.0.1:3101/metrics>

Vérifier : `kubectl get pods -n logging`

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n logging
NAME                                READY   STATUS    RESTARTS   AGE
loki-0                              2/2    Running   0           20m
loki-canary-9c475                   1/1    Running   0           20m
loki-canary-krk22                   1/1    Running   0           20m
loki-canary-zqq2f                   1/1    Running   0           20m
loki-gateway-7bdf4bbf46-fp698      1/1    Running   0           20m
loki-minio-0                        1/1    Running   0           20m
promtail-hmtz8                      1/1    Running   0           57s
promtail-nj674                     1/1    Running   0           57s
promtail-vpz69                     1/1    Running   0           57s
henri@Henri:~/labs/sre-kubernetes-lab$
```

Vérifier les logs promtail

7.8 Générer des logs

Créer le fichier : `/logging/log-generator.yaml`

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: log-generator
  namespace: demo-app

spec:
  replicas: 1

  selector:
    matchLabels:
      app: log-generator

  template:
    metadata:
      labels:
        app: log-generator

    spec:
      containers:
        - name: log-generator
          image: busybox

          command:
            - /bin/sh
            - -c
            - |
              while true; do
                echo "$(date) INFO Application running"
```

sleep 2 done

7.9 Déployer

kubectl apply -f logging/log-generator.yaml

7.10 Vérifier les log K8s

kubectl logs -n demo-app deployment/log-generator

On devrais voir : INFO Application running

```
henri@henri: ~/labs/sre-kubernetes-lab$ kubectl logs -n demo-app deployment/log-generator
Sat May 9 01:04:53 UTC 2026 INFO Application running
Sat May 9 01:04:55 UTC 2026 INFO Application running
Sat May 9 01:04:57 UTC 2026 INFO Application running
Sat May 9 01:04:59 UTC 2026 INFO Application running
```

7.11 Vérifier que Loki reçoit les logs

Aller dans Grafana

Ajouter se ce n'est déjà fait :

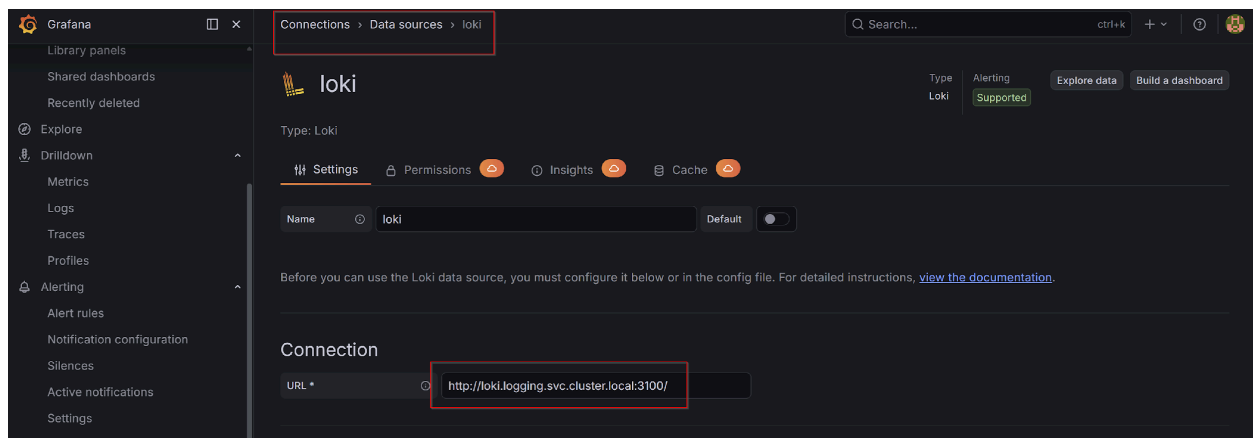
Connections

→ Data Sources

→ Add Data Source

→ Loki

Saisir l'URL de Loki : <http://loki.logging:3100> ou bien <http://loki-gateway.logging.svc.cluster.local>



Puis pour continuer les tests :

kubectl port-forward svc/<service-name> -n logging 3100:3100

Dans notre lab service-name = loki

Donc :

kubectl port-forward svc/loki -n logging 3100:3100

puis :

curl <http://localhost:3100/ready>

si on voit eady . c'Est OK

7.12 Tester les logs dans Grafana

Dans : Explore

Requête : {job=~".+"}

requête

ou :

{app="log-generator"}

Ou :

Logs Traefik

{namespace="kube-system"}

7.13 Erreur frequente avec Grafana/loki

J'Ai constaté qu'on a e message de waring : "too many unhealthy instance in the ring, dans Grafana et qu'on utilise la requête : {namespace="demo-app"}

Solution :

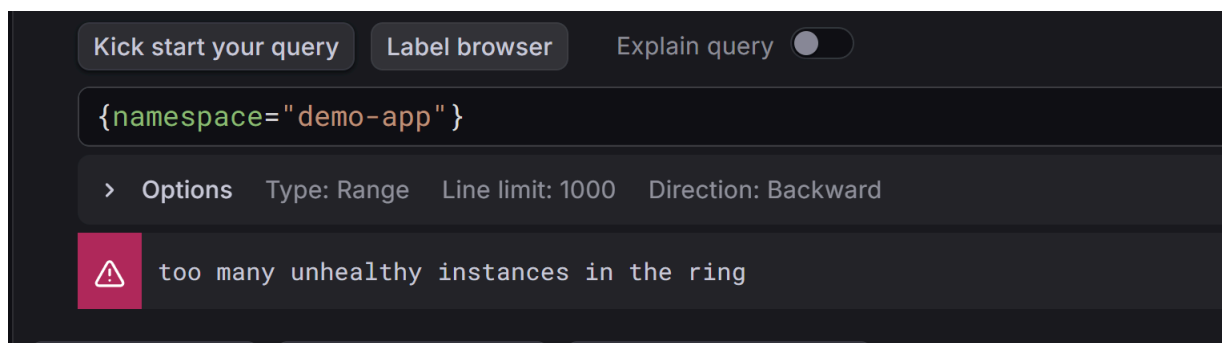
Supprimer le pod loki : kubectl delete pod loki-0 -n logging

Attendre, Le StatefulSet va le recréer automatiquement.

kubectl get pods -n logging -w

```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n logging -w
NAME          READY   STATUS    RESTARTS   AGE
loki-0        2/2     Running   0           54s
```

Verifier que loki-0 2/2 running



8 État du cluster au repos (avant charge)

8.1 État du cluster avant les tests de charge

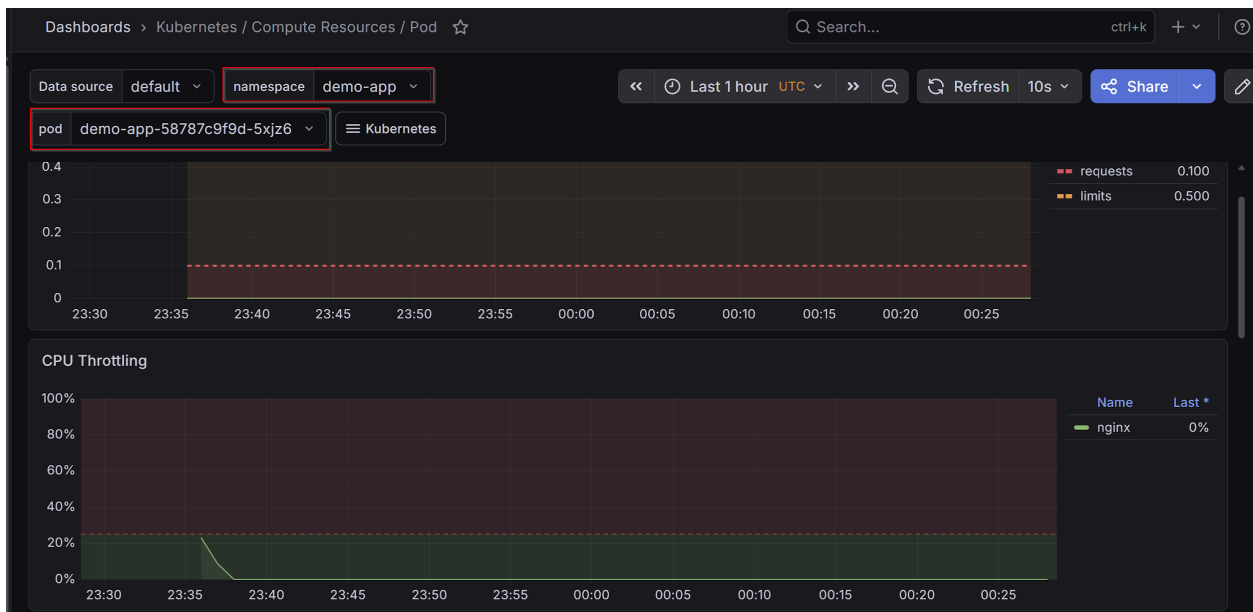
Il n'y a au départ que 2 pods dans le cluster

```
henri@Henri:~/labs/sre-kubernetes-lab$
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n demo-app
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-58787c9f9d-5xjz6          1/1    Running   0          55m
demo-app-58787c9f9d-vtqgd          1/1    Running   0          55m
henri@Henri:~/labs/sre-kubernetes-lab$
```

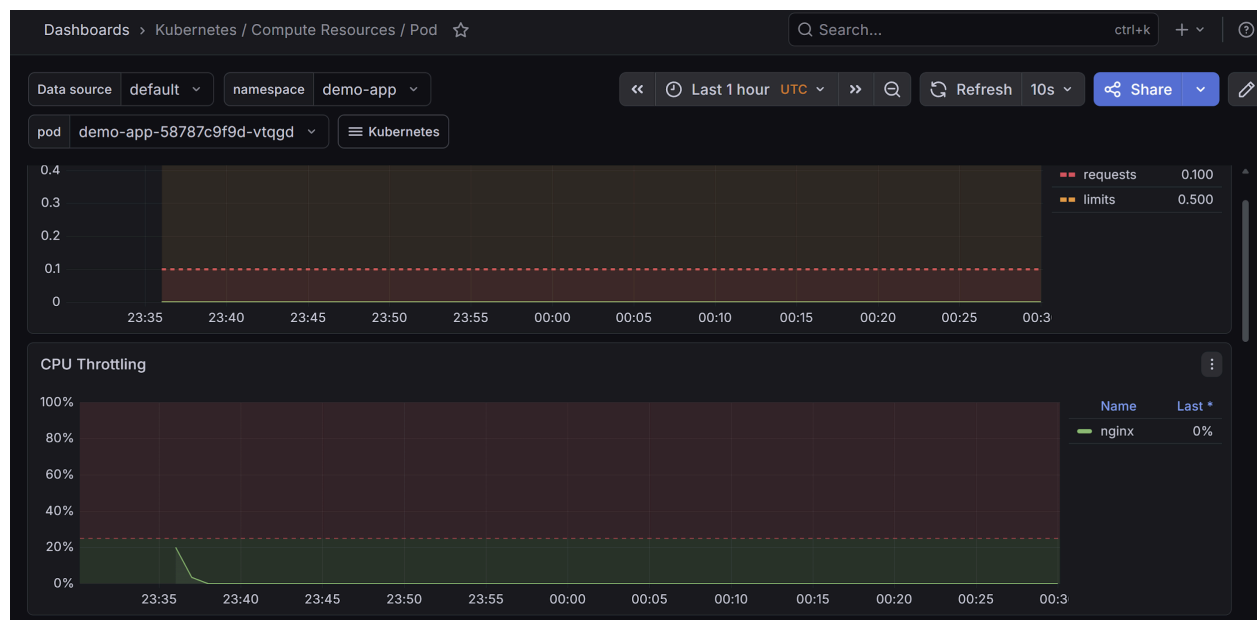
Charge CPU sur le cluster : demo-app :



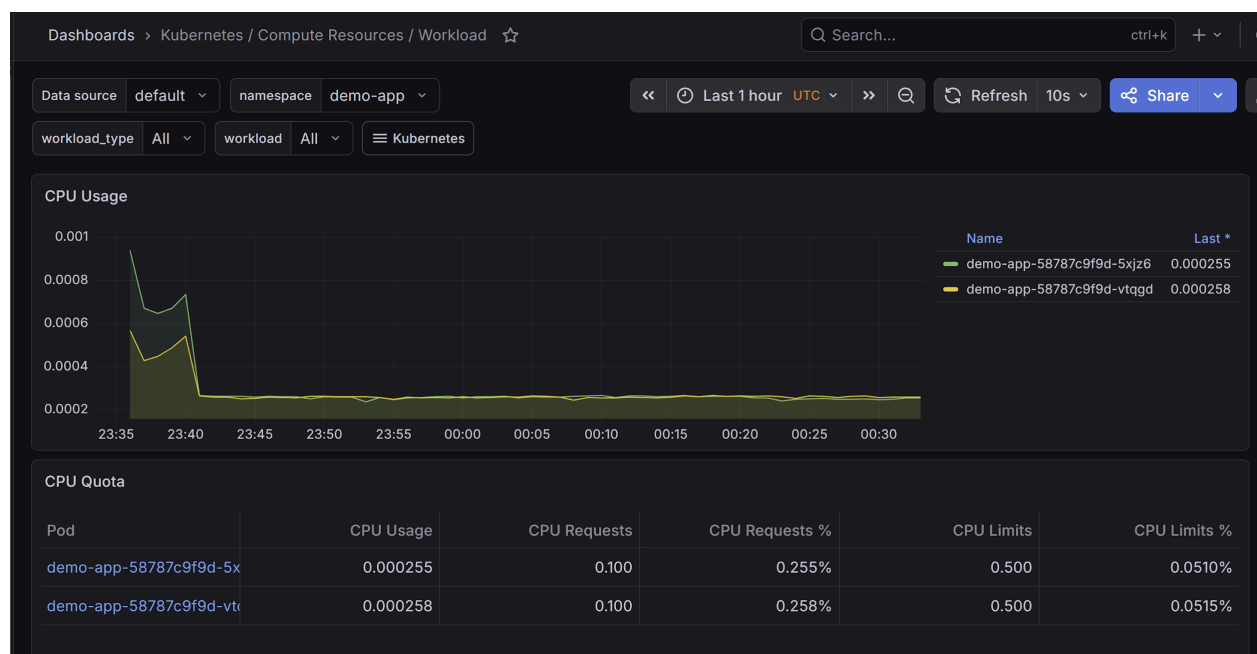
Charge CPU sur le pod 1 :



Charge CPU sur le pod 2 :



Pour l'ensemble des 2 pods :



8.2 k9s – vue pods

Vue K9s

Vue de l'ensemble des pods

Context: k3d-sre-lab [RW]
Cluster: k3d-sre-lab
User: admin@k3d-sre-lab
K8s Rev: v0.50.10
K8s Rev: v1.31.5+k3s1
CPU: 0%
MEM: 8%

<0> all
<1> default
<a>
<ctrl-d>
<d>
<o>
<?>
<shift-j>

Attach
Delete
Describe
Edit
Help
Jump Owner

<ctrl-k>
<l>
<p>
<shift-f>
<+>
<->

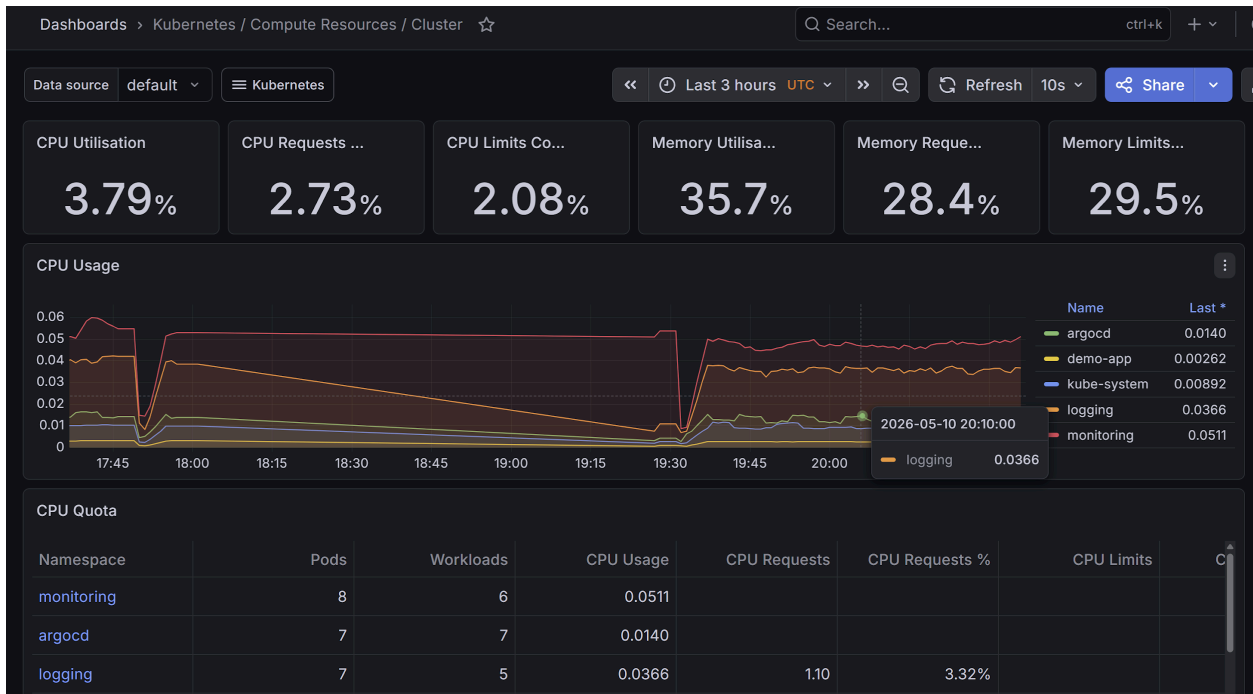
Kill
Logs
Logs Previous
Port-Forward
Sanitize
Shell

<o> Show Node
<f> Show PortForward
<t> Transfer
<m> Warp To Namespace
<y> YAML

NAMESPACE	NAME	PF	READY	STATUS	RESTARTS	CPU	%CPU/R	%CPU/L	MEM	%MEM/R	%MEM/L	IP	NODE
argocd	argocd-application-controller-0	●	1/1	Running	2	4	n/a	n/a	167	n/a	n/a	10.42.0.67	k3d-sre-lab-a
argocd	argocd-applicationset-controller-6bdfccc9c6-8z6gz	●	1/1	Running	1	1	n/a	n/a	20	n/a	n/a	10.42.2.85	k3d-sre-lab-a
argocd	argocd-dex-server-cdd7bd88f-nnrgl	●	1/1	Running	1	1	n/a	n/a	17	n/a	n/a	10.42.2.89	k3d-sre-lab-a
argocd	argocd-notifications-controller-5b59cc856f-xnhqm	●	1/1	Running	2	1	n/a	n/a	17	n/a	n/a	10.42.0.74	k3d-sre-lab-a
argocd	argocd-repo-server-548bf6d4d7-jrnbj	●	1/1	Running	1	5	n/a	n/a	6	n/a	n/a	10.42.0.69	k3d-sre-lab-a
argocd	argocd-repo-server-86998b885-f96np	●	1/1	Running	2	1	n/a	n/a	42	n/a	n/a	10.42.2.86	k3d-sre-lab-a
argocd	argocd-server-56b6d45cd6-fg2n9	●	1/1	Running	2	1	n/a	n/a	23	n/a	n/a	10.42.2.81	k3d-sre-lab-a
demo-app	demo-app-69b4cc4778-2msxg	●	1/1	Running	1	1	20	0	15	12	3	10.42.2.80	k3d-sre-lab-a
demo-app	demo-app-69b4cc4778-6xk7l	●	1/1	Running	0	1	20	0	13	10	2	10.42.2.91	k3d-sre-lab-a
demo-app	log-generator-76786dd7fc-9622l	●	1/1	Running	2	3	n/a	n/a	3	n/a	n/a	10.42.0.64	k3d-sre-lab-a
kube-system	coredns-ccb96694c-6s6fx	●	1/1	Running	1	3	3	n/a	23	32	13	10.42.0.75	k3d-sre-lab-a
kube-system	helm-install-traefik-crd-zssfn	●	0/1	Completed	0	0	n/a	n/a	0	n/a	n/a	n/a	k3d-sre-lab-a
kube-system	helm-install-traefik-l6n6t	●	0/1	Completed	1	0	n/a	n/a	0	n/a	n/a	n/a	k3d-sre-lab-a
kube-system	local-path-provisioner-5cf85fd84d-8wt4u	●	1/1	Running	1	1	n/a	n/a	8	n/a	n/a	10.42.2.79	k3d-sre-lab-a
kube-system	metrics-server-5c9df4bf4b-xzsqr	●	1/1	Running	0	5	5	n/a	25	35	n/a	10.42.2.93	k3d-sre-lab-a
kube-system	svclb-traefik-51d1ec19-ccrwn	●	2/2	Running	4	0	n/a	n/a	1	n/a	n/a	10.42.0.62	k3d-sre-lab-a
kube-system	svclb-traefik-51d1ec19-t5p8x	●	2/2	Running	4	0	n/a	n/a	2	n/a	n/a	10.42.2.76	k3d-sre-lab-a
kube-system	svclb-traefik-51d1ec19-xzwj5	●	2/2	Running	4	0	n/a	n/a	2	n/a	n/a	10.42.1.41	k3d-sre-lab-s
kube-system	traefik-5d45fc8cc9-zbc94	●	1/1	Running	0	1	n/a	n/a	31	n/a	n/a	10.42.1.70	k3d-sre-lab-s
logging	loki-0	●	2/2	Running	2	7	n/a	n/a	163	n/a	n/a	10.42.2.92	k3d-sre-lab-a
logging	loki-chunks-cache-0	●	2/2	Running	0	2	0	n/a	9	0	0	10.42.2.94	k3d-sre-lab-a
logging	loki-minio-0	●	1/1	Running	0	1	1	n/a	185	82	n/a	10.42.1.43	k3d-sre-lab-s
logging	loki-results-cache-0	●	2/2	Running	2	3	0	n/a	11	0	0	10.42.0.68	k3d-sre-lab-a
logging	promtail-7qrdz	●	1/1	Running	1	7	n/a	n/a	37	n/a	n/a	10.42.0.63	k3d-sre-lab-a
logging	promtail-nst9s	●	1/1	Running	1	6	n/a	n/a	24	n/a	n/a	10.42.1.42	k3d-sre-lab-a
logging	promtail-wlmm	●	1/1	Running	1	9	n/a	n/a	47	n/a	n/a	10.42.2.83	k3d-sre-lab-a
monitoring	alertmanager-monitoring-kube-prometheus-alertmanager-0	●	2/2	Running	4	1	n/a	n/a	34	17	n/a	10.42.2.87	k3d-sre-lab-a
monitoring	monitoring-grafana-74598f477b-xhcfz	●	3/3	Running	3	10	n/a	n/a	356	n/a	n/a	10.42.2.88	k3d-sre-lab-a
monitoring	monitoring-kube-prometheus-operator-6c8c5bb9b-vmz7d	●	1/1	Running	2	4	n/a	n/a	23	n/a	n/a	10.42.2.84	k3d-sre-lab-a
monitoring	monitoring-kube-state-metrics-64485fd97-8bmjv	●	1/1	Running	2	1	n/a	n/a	20	n/a	n/a	10.42.0.65	k3d-sre-lab-a
monitoring	monitoring-prometheus-node-exporter-hvdc8	●	1/1	Running	2	3	n/a	n/a	10	n/a	n/a	172.21.0.4	k3d-sre-lab-a
monitoring	monitoring-prometheus-node-exporter-mxplk	●	1/1	Running	2	1	n/a	n/a	10	n/a	n/a	172.21.0.3	k3d-sre-lab-s
monitoring	monitoring-prometheus-node-exporter-qkvkk	●	1/1	Running	2	4	n/a	n/a	11	n/a	n/a	172.21.0.5	k3d-sre-lab-a
monitoring	prometheus-monitoring-kube-prometheus-prometheus-0	●	2/2	Running	4	36	n/a	n/a	441	n/a	n/a	10.42.0.73	k3d-sre-lab-a

8.3 Grafana – Vue cluster

Vue k9s
Vue de l'ensemble des namespaces du cluster
La charge CPU du cluster ne dépasse pas en moyenne 6% CPU (0.06)



8.4 Analyse des differends workloads

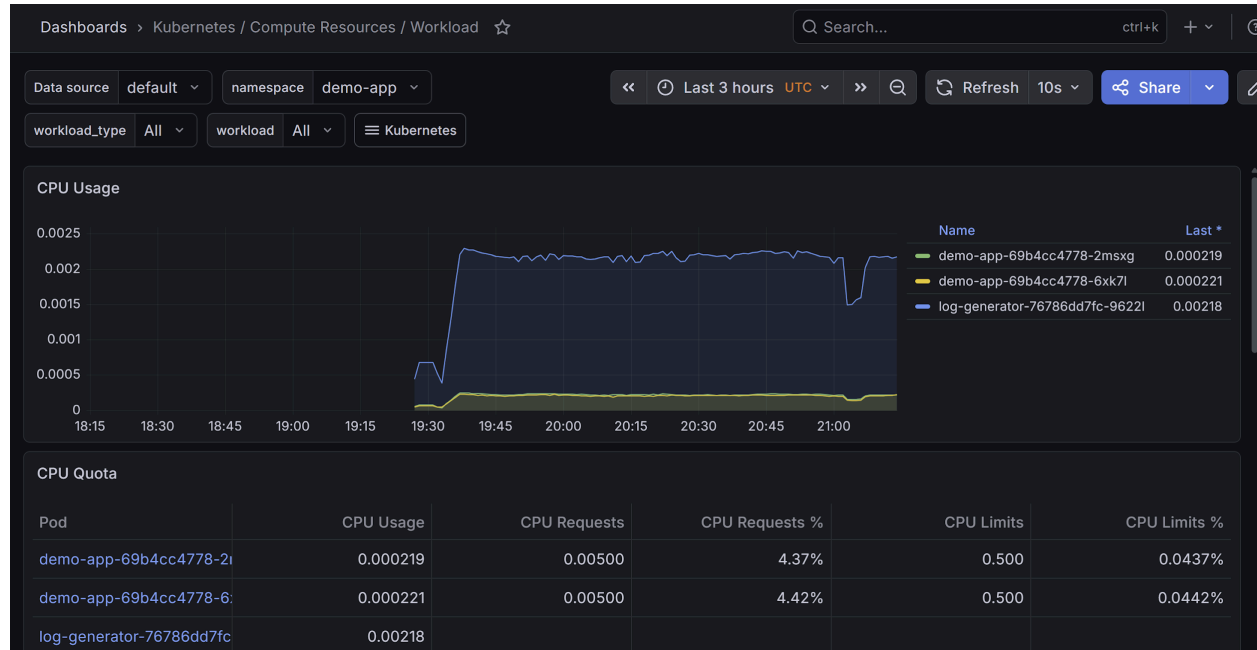
Vue : all workloads

Les courbes des deux workloads existants sont superposés , dans le meme graphe

On voit le workload des pods applicatifs. Ici on a seulement 2 pods (en dehors des test de charge , le cluster est au repos, le nombre de pods est identique à ce qui est défini dans le déploiement initial)

Pods intitulés : demo-app-xxxxxxx , qui sont dans le namespace : demo-app

On voit aussi le workload du pod de génération des logs , qui fonctionne dans le même namespace : demo-app. (courbe en bleu). Ce pod génère un trafic constant , qui est proche de 0.0025 CPU.



9 Load Testing

9.1 Test 1 (K6)

9.1.1 Installer K6

```
henri@Henri:~/labs/sre-kubernetes-lab$ k6
Command 'k6' not found, but can be installed with:
sudo snap install k6
henri@Henri:~/labs/sre-kubernetes-lab$ sudo snap install k6
[sudo] password for henri:
2026-05-08T17:29:23-04:00 INFO Waiting for automatic snapd restart...
Automatically connect eligible plugs and slots of snap "snapd"
```

9.1.2 fichier de test

Load-testing/k6-loadtest.js

```
import http from 'k6/http';

export const options = {
  vus: 200,
  duration: '5m',
```

```
};

export default function () {
  http.get('http://localhost:8080', {
    headers: {
      Host: 'demo.local',
    },
  });
};
}
```

9.1.3 Run:

k6 run k6-loadtest.js

k6 run load-testing/k6-loadtest.js

```
henri@Henri:~/labs/sre-kubernetes-lab$ cd load-testing
henri@Henri:~/labs/sre-kubernetes-lab/load-testing$ k6 run k6-loadtest.js
```



```

  execution: local
    script: k6-loadtest.js
    output: -

  scenarios: (100.00%) 1 scenario, 20 max VUs, 2m30s max duration (incl. graceful stop):
    * default: 20 looping VUs for 2m0s (gracefulStop: 30s)

running (0m09.0s), 20/20 VUs, 175 complete and 0 interrupted iterations
default  [=>-----] 20 VUs  0m09.1s/2m0s
```

9.1.4 Observer scaling :

kubectl get hpa -w

au départ

kubectl get pods -n demo-app -w

```
henri@Henri:~/mnt/c/Users/henri$ kubectl get pods -n demo-app -w
```

NAME	READY	STATUS	RESTARTS	AGE
demo-app-69b4cc4778-2msxg	1/1	Running	1 (128m ago)	13h
demo-app-69b4cc4778-6xk7l	1/1	Running	0	12h
log-generator-76786dd7fc-9622l	1/1	Running	2 (128m ago)	39h

Puis en cours de charge K6 :

Et aussi :

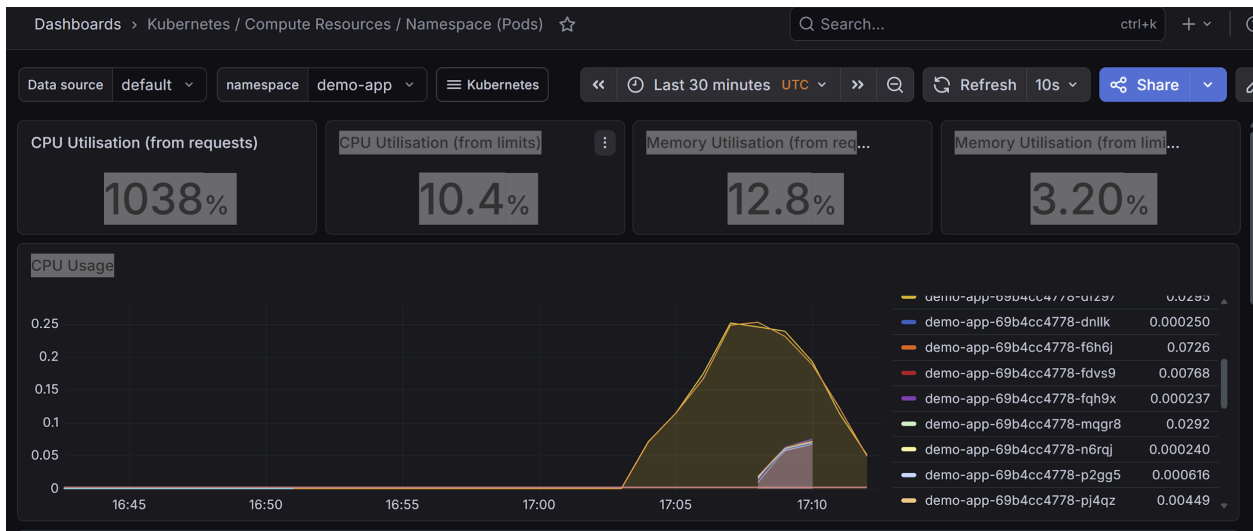
```
henri@henri:/mnt/c/Users/henri$ kubectl get hpa -w -n demo-app
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
demo-app-hpa	Deployment/demo-app	cpu: 20%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 4500%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 10270%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 8370%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 8370%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 20%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 20%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 1040%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 5640%/50%	2	10	6	41h
demo-app-hpa	Deployment/demo-app	cpu: 9990%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 3070%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 20%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 6820%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 7320%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 10760%/50%	2	10	2	41h
demo-app-hpa	Deployment/demo-app	cpu: 10910%/50%	2	10	6	41h
demo-app-hpa	Deployment/demo-app	cpu: 9970%/50%	2	10	10	41h
demo-app-hpa	Deployment/demo-app	cpu: 6846%/50%	2	10	10	41h
demo-app-hpa	Deployment/demo-app	cpu: 3440%/50%	2	10	2	41h

verifier dans Grafana :

Grafana sur les dernières 30 minutes :

Dasn Grafana lors du test k6 , on constate un pic a 0.25 de charge CPU. Soit 25% charge CPU



Analyse des résultats :

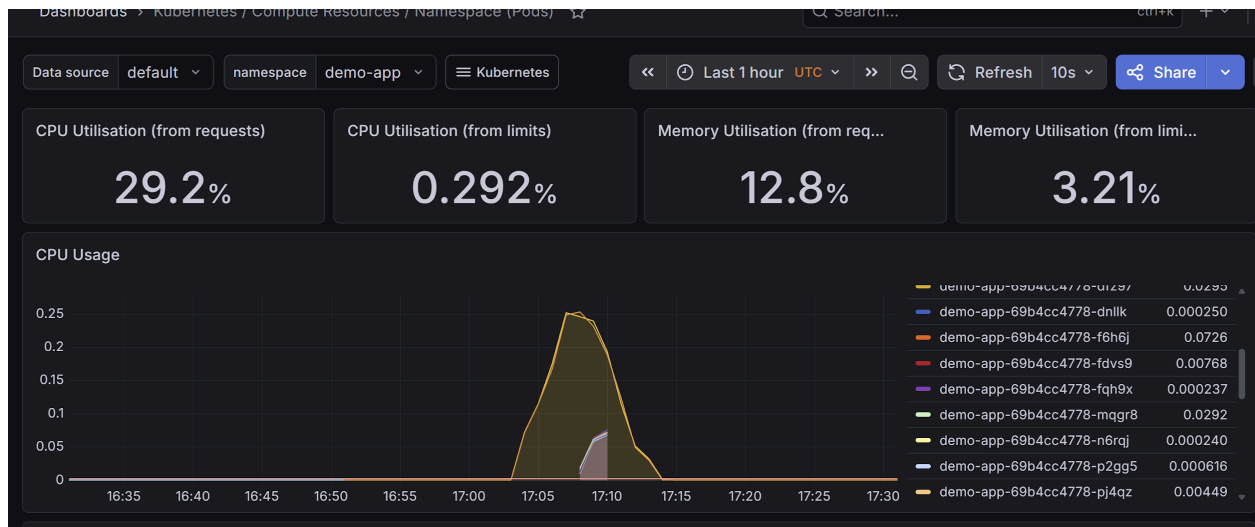
- ✓ forte montée CPU
- ✓ réaction du HPA
- ✓ scale up jusqu'à 10 pods
- ✓ scale down ensuite
- ✓ métriques Grafana cohérentes

Graphe CPU Usage de Gafana , Le pic : 0.25 en ordonnées du graphe signifie :

$0.25 \times 100 = 25\%$ CPU réel Donc :un quart de CPU complet utilisé , ce qui est cohérent avec un test k6 agressif

Pourquoi 1038% paraît énorme Parce que : requests: cpu: 25m, probablement. Donc :Si consommation réelle :250m alors : $\frac{250m}{25m} \times 100 = 1000\%$
exactement ce que Grafana affiche.

Grafana sur la dernière heure :



9.2 Test 2 (hey)

`kubectrl port-forward svc/demo-app 8888:80 -n demo-app`

`hey -z 5m -c 500 http://localhost:8888`

-z 5m : durée du test = 5 minutes

-c 500 : nombre de connectins simultanées

On observe alors que HPA fonctionne , le cluster scale de 2 à 10 replicas. On passe de 2 à 10 pods
Cf figure ci-dessous.

```
henri@Henri:/mnt/c/Users/henri$ kubectl get pods -n demo-app -w
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-69b4cc4778-548wv           1/1     Running   0           2m7s
demo-app-69b4cc4778-6mfpj           1/1     Running   0           114s
demo-app-69b4cc4778-94twr           1/1     Running   0           114s
demo-app-69b4cc4778-gmvtd           1/1     Running   0           2m7s
demo-app-69b4cc4778-hf74z           1/1     Running   0           99s
demo-app-69b4cc4778-k6ql2           1/1     Running   0           114s
demo-app-69b4cc4778-lvvgb           1/1     Running   0           114s
demo-app-69b4cc4778-q9sbd           1/1     Running   0           10m
demo-app-69b4cc4778-qdxkq           1/1     Running   0           99s
demo-app-69b4cc4778-vswwg           1/1     Running   0           7m4s
log-generator-76786dd7fc-9622l      1/1     Running   1 (11h ago)  26h
```

Resultats du test hey

```
henri@Henri:/mnt/c/Users/henri$ hey -z 5m -c 500 http://localhost:8888
```

Summary:

Total:	300.0701 secs
Slowest:	0.5667 secs
Fastest:	0.0014 secs
Average:	0.1500 secs
Requests/sec:	7788.6412

Total data: 2094075648 bytes
Size/request: 2094 bytes

Response time histogram:

0.001 [1]		
0.058 [409240]		=====
0.114 [571740]		=====
0.171 [181113]		
0.228 [548]		
0.284 [16]		
0.341 [67]		
0.397 [75]		
0.454 [62]		
0.510 [130]		
0.567 [8]		

```
Response time histogram:
0.001 [1] |
0.058 [409240] |=====
0.114 [571740] |=====
0.171 [18113] |■
0.228 [548] |
0.284 [16] |
0.341 [67] |
0.397 [75] |
0.454 [62] |
0.510 [130] |
0.567 [8] |
```

```

Latency distribution:
  10% in 0.0306 secs
  25% in 0.0375 secs
  50% in 0.0679 secs
  75% in 0.0852 secs
  90% in 0.0982 secs
  95% in 0.1054 secs
  99% in 0.1206 secs

Details (average, fastest, slowest):
  DNS+dialup:    0.0000 secs, 0.0014 secs, 0.5667 secs
  DNS-lookup:    0.0000 secs, 0.0000 secs, 0.0461 secs
  req write:      0.0000 secs, 0.0000 secs, 0.0101 secs
  resp wait:      0.1463 secs, 0.0013 secs, 0.5163 secs
  resp read:      0.0036 secs, 0.0000 secs, 0.1415 secs

Status code distribution:
  [200] 1000000 responses

```

```
10% in 0.0306 secs
25% in 0.0375 secs
50% in 0.0679 secs
75% in 0.0852 secs
90% in 0.0982 secs
95% in 0.1054 secs
99% in 0.1206 secs
```

```
Details (average, fastest, slowest):
  DNS+dialup:    0.0000 secs, 0.0014 secs, 0.5667 secs
  DNS-lookup:    0.0000 secs, 0.0000 secs, 0.0461 secs
  req write:     0.0000 secs, 0.0000 secs, 0.0101 secs
  resp wait:     0.1463 secs, 0.0013 secs, 0.5163 secs
  resp read:     0.0036 secs, 0.0000 secs, 0.1415 secs

Status code distribution:
[200] 1000000 responses
```

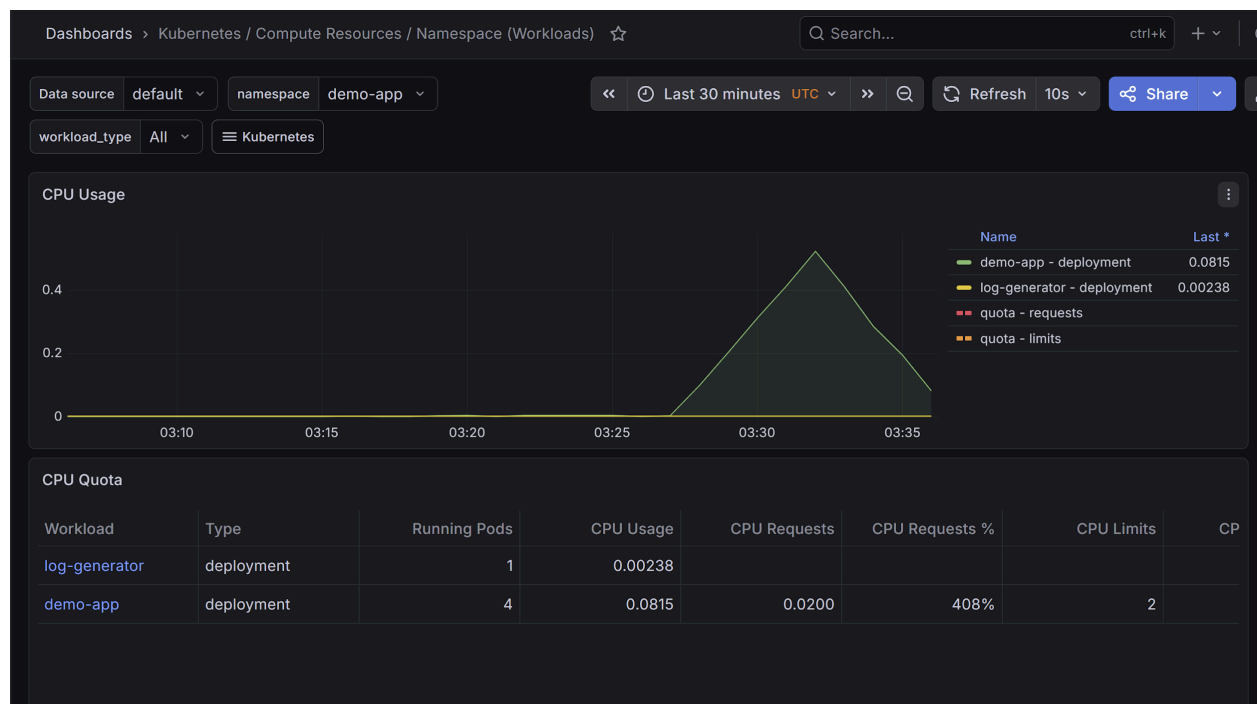
```
DNS+dialup: 0.0000 secs, 0.0014 secs, 0.5667 secs
DNS-lookup: 0.0000 secs, 0.0000 secs, 0.0461 secs
req write: 0.0000 secs, 0.0000 secs, 0.0101 secs
resp wait: 0.1463 secs, 0.0013 secs, 0.5163 secs
resp read: 0.0036 secs, 0.0000 secs, 0.1415 secs
```

```
Status code distribution:
  [200] 1000000 responses
```

```
[200] 1000000 responses
```

Résutats dans Grafana :

Le test de charge avec hey a durée 5 minutes et pendant ce laps de temps la charge CPU pour le cluster (les 2 pods qui tournent dans demo-app)) est montée à 0.4, puis elle est retombée à 0.1 a la fin du test



Le nombre de pods a scélé, il est passé de 2 à 10 .

L'utilisation CPU pour chaque pod du namespace est représenté dans la figure ci-dessous.

On constate dans le graphe Grafana, un pic lors de la durée du test, pour lequel on voit que le CPU a grimpé jusqu'à 0.4

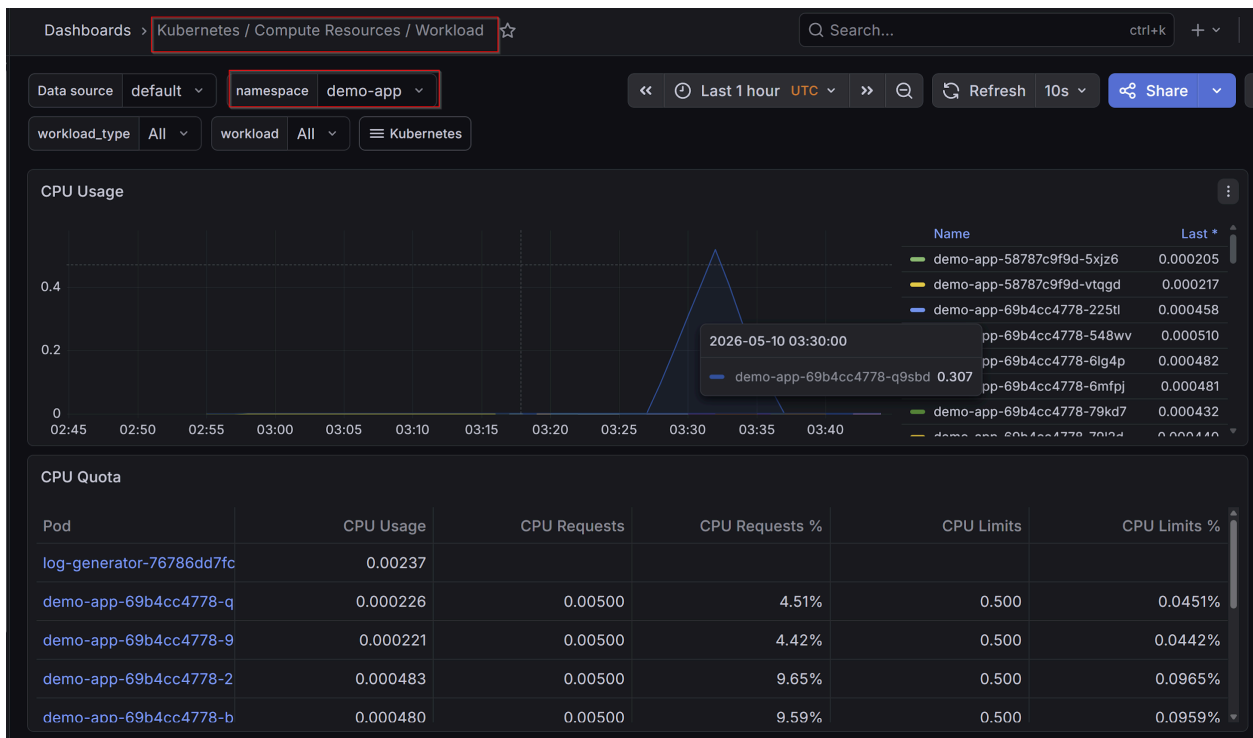
Rappel : 0.4 CPU cores=400m (400 miliCPU) c'est-à-dire : 40% d'un cœur CPU ou 400 milliCPU.

Donc la charge est montée jusqu'à 40 % , puis est retombée à 1m (soit 0.1%)

Ce qu'on constate également avec la commande `kubectl top nodes`, dans la figure ci-dessous , dans laquelle on voit les 10 pods qui tournent chacun avec seulement 1m (0.1%)

`kubectl top pods -n demo-app`

```
henri@Henri:/mnt/c/Users/henri$ kubectl top pods -n demo-app
NAME                                CPU(cores)   MEMORY(bytes)
demo-app-69b4cc4778-2msxg           1m           13Mi
demo-app-69b4cc4778-4twh5           1m           13Mi
demo-app-69b4cc4778-94twr           1m           13Mi
demo-app-69b4cc4778-dnllk           1m           13Mi
demo-app-69b4cc4778-fqh9x           1m           13Mi
demo-app-69b4cc4778-n6rqj           1m           13Mi
demo-app-69b4cc4778-q9sbd           1m           13Mi
demo-app-69b4cc4778-shvvc           1m           13Mi
demo-app-69b4cc4778-sjm2t           1m           13Mi
demo-app-69b4cc4778-xzl57           1m           13Mi
log-generator-76786dd7fc-9622l      3m           3Mi
henri@Henri:/mnt/c/Users/henri$
```



9.3 Test K6 plus agressif

```
import http from 'k6/http';
```

```
export const options = {
  stages: [
    { duration: '1m', target: 100 },
    { duration: '2m', target: 500 },
    { duration: '2m', target: 1000 },
    { duration: '1m', target: 0 },
  ]
}
```

```

},
};

export default function () {
  http.get('http://localhost:8080', {
    headers: {
      Host: 'demo.local',
    },
  });
}

```

9.4 Tests Logs Lki

Observer les logs dans Grafana suite aux tests 1 & 2 précédents
 Si on arrive a faire fonctionner Loki ...!!!!
 Ce qui n'a pas été le cas dans ce lab.

10 Incident Simulation

10.1 Ingress Failure

On teste la résilience du système. La destruction d'un pod , entraine sa recreation immédiate
 On veut tester la résilience d'un pod traefik.

kubectl get pod -n kube-system.

```

henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n kube-system -w
NAME                                READY    STATUS    RESTARTS    AGE
coredns-ccb96694c-6s6fx             1/1      Running   1 (4h49m ago)  41h
helm-install-traefik-crd-zssfn       0/1      Completed 0            2d3h
helm-install-traefik-l6n6t          0/1      Completed 1            2d3h
local-path-provisioner-5cf85fd84d-8wt44 1/1      Running   1 (4h49m ago)  41h
metrics-server-5c9df4bf4b-xzsqr     1/1      Running   0            15h
svclb-traefik-51d1ec19-ccrwn        2/2      Running   4 (4h49m ago)  2d3h
svclb-traefik-51d1ec19-t5p8x        2/2      Running   4 (4h49m ago)  2d3h
svclb-traefik-51d1ec19-xzwj5        2/2      Running   4 (4h49m ago)  2d3h
traefik-5d45fc8cc9-j78bk            1/1      Running   0            12s

```

Puis :

kubectl delete pod traefik-5d45fc8cc9-j78bk -n kube-system

on constate : que le pod traefik a été recrée par le système.

Le pod traefik-5d45fc8cc9-zbc94 a été recrée. Pod identique , mis à part le nom


```
henri@Henri:~/labs/sre-kubernetes-lab$ kubectl get pods -n kube-system -w
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-ccb96694c-6s6fx	1/1	Running	1 (5h1m ago)	41h
helm-install-traefik-crd-zssfn	0/1	Completed	0	2d4h
helm-install-traefik-l6n6t	0/1	Completed	1	2d4h
local-path-provisioner-5cf85fd84d-8wt44	1/1	Running	1 (5h1m ago)	41h
metrics-server-5c9df4bf4b-xzsqr	1/1	Running	0	15h
svclb-traefik-51d1ec19-ccrwn	2/2	Running	4 (5h1m ago)	2d4h
svclb-traefik-51d1ec19-t5p8x	2/2	Running	4 (5h1m ago)	2d4h
svclb-traefik-51d1ec19-xzwj5	2/2	Running	4 (5h2m ago)	2d4h
traefik-5d45fc8cc9-zbc94	1/1	Running	0	6s